

The web

`std.http` has two halves: fetching things, and serving them. Both are built on `std.net`, which is plain sockets if you ever need to go lower.

Serving

A server is a port and a function. The function is handed a request and gives back a reply.

```
use std.io
use std.http

fn handle(request: Map<Text, Any>) -> Map<Text, Any> {
  give ["status": 200, "type": "text/plain", "body": "hello from SPRFST"]
}

fn main() {
  io.say("listening on http://localhost:8080")
  http.serve(8080, handle)
}
```

`http.serve` does not come back—it accepts connections until the program is stopped, or until some other part of the program calls `http.stop()`.

What is in a request

Key	Example
method	GET
path	/notes/3
query	tag=work&sort=new
headers	a map of lowercase header names to values
body	the request body as text

What a reply may contain

Key	Default	Meaning
status	200	the status code
type	text/html; charset=utf-8	the content type
body	empty	the text to send

A plain `Text` reply also works, and is treated as HTML with status 200.

Routing is just code

Because the handler is an ordinary function, you can call it directly—which means a web application can be tested without a socket in sight.

```
use std.io
use std.json

fn handle(request: Map<Text, Any>) -> Map<Text, Any> {
    let path = request.get("path") ?? "/"
    let method = request.get("method") ?? "GET"

    if path == "/" {
        give ["status": 200, "body": "<h1>SPRFST</h1>"]
    }
    if path == "/health" {
        give ["status": 200, "type": "application/json",
            "body": json.write(["ok": true, "version": "0.1.0"])]
    }
    if path == "/echo" and method == "POST" {
        give ["status": 200, "type": "text/plain",
            "body": request.get("body") ?? ""]
    }
    give ["status": 404, "type": "text/plain", "body": "no page at {path}"]
}

fn try_it(method: Text, path: Text, body: Text) {
    let reply = handle(["method": method, "path": path, "body": body])
    io.say("{method} {path} -> {reply.get("status")} {reply.get("body")}")
}

fn main() {
    try_it("GET", "/", "")
    try_it("GET", "/health", "")
    try_it("POST", "/echo", "ping")
    try_it("GET", "/nowhere", "")
}
```

Run that and you have tested four routes in a few milliseconds. The same `handle` passed to `http.serve` is the live site.

Building pages

There is no template language. Text interpolation is the template language, and `{{` and `}}` give you literal braces for CSS:

```
use std.io

fn page(title: Text, body: Text) -> Text {
    give "<!doctype html><meta charset=\"utf-8\"><title>{title}</title>" +
        "<style>body{{background:#0b0b0d;color:#f2f2f0;font:16px system-ui}}" +
        "h1{{color:#ffa136}}</style>" + body
}
```

```

}

fn main() {
    io.say(page("Notes", "<h1>Notes</h1><p>Nothing yet.</p>").len().to_text() + " bytes")
}

```

Fetching

```

use std.io
use std.http
use std.json

fn main() {
    let text = http.get("http://example.com/api/items") ?? ""
    let data = json.parse(text)
    io.say("{data}")

    let reply = http.post("http://example.com/api/items", json.write(["name": "pen"]))
    io.say("{reply ?? \"no reply\"}")
}

```

Both give back `nil` when the request fails, so `??` is the natural way to carry on.

A limitation worth knowing. `http.get` and `http.post` speak plain HTTP. There is no TLS, so an `https://` address returns `nil`. Until that lands, fetch through a local proxy or use `sys.run("curl ...")`.

Query strings and encoding

```

use std.io
use std.http

fn main() {
    let q = http.url_encode("tag=work & urgent")
    io.say("encoded {q}")
    io.say("decoded {http.url_decode(q)}")

    var fields: Map<Text, Text> = [:]
    for pair in "tag=work&sort=new".split("&") {
        let bits = pair.split("=")
        if bits.len() == 2 { fields.set(bits[0], http.url_decode(bits[1])) }
    }
    io.say("{fields}")
}

```

The example to read next

`examples/19-web-server.spf` is a complete site: a styled home page, a JSON health endpoint, a clock, and a 404 page, in sixty lines.

```
sprfst run examples/19-web-server.spf
```

Exercise

Add a `/notes` route to the routing example that gives back a JSON list, and a `POST /notes` that adds to it. Keep the notes in a module level `var` and test both by calling `handle` directly.